

# Ders 2 Çalışma Özeti

## Ders 2 Çalışma Özeti

### Genel Konular

- Kontrol akışını ele geçirme saldırıları
  - Saldırganın amacı hedef sistemde çalışan bir servisin veya programın kontrol akışını değiştirerek keyfi kod çalıştırmaktır.
  - Web, mail, DNS gibi sürekli çalışan servisler doğal saldırı vektörleri oluşturur; ancak kısa süreli çalışan programlarda da benzer zafiyetler bulunabilir.
- Temel zafiyet türleri
  - Buffer overflow, integer overflow, format string vulnerability, use after free ve double free kontrol akışını değiştirmede kullanılabilecek zafiyet türleri olarak ele alınır.
  - Buffer overflow genellikle en temel ve uygulanabilir örnek olarak anlatılır; format string zafiyetleri daha çok hassas veri sızdırma veya sınırlı kontrol elde etme amacıyla da kullanılabilir.
- Buffer overflow tarihçesi
  - Unix finger servisine yönelik ilk solucan örnekleri, bellek taşması zafiyetlerinin ağ üzerinde yayılabilen saldırılara dönüşebileceğini gösterir.
  - Bir zafiyetin etkisi yalnızca zafiyet sayısıyla değil, zafiyetli yazılımın kaç sistemde çalıştığıyla belirlenir.
- C/C++ ve çalışma zamanı ortamı
  - İşletim sistemleri, sistem kütüphaneleri ve çalışma zamanı bileşenlerinin büyük bölümü C/C++ ile yazıldığı için bellek güvenliği zafiyetleri güncel sistemleri etkilemeye devam eder.
  - Daha güvenli diller kullanılsa bile alt katmanda C/C++ kütüphaneleri bulunduğu risk tamamen ortadan kalkmaz.
- Bellek düzeni ve stack frame
  - 32 bit Linux örneği üzerinden proses adres alanı, stack, heap, shared library bölgesi ve executable bölgesi açıklanır.
  - Stack frame; argümanlar, dönüş adresi, stack frame pointer, yerel değişkenler ve saklanan register değerlerinden oluşur.

### Hocanın Özellikle Vurguladığı Kısımlar

- Uygulama değil kontrol akışı ele geçirilir
  - Saldırgan programın tasarlanan davranışını bozarak kendi istediği kodu çalıştırır.
- Attack vector input kabul eden noktadır
  - Bir fonksiyon veya servis dışı girdiyi kabul etmiyorsa o noktadan saldırı yapmak mümkün değildir.
- Stack düzeni saldırıyı anlamak için kritiktir
  - Dönüş adresinin stack üzerinde tutulması, taşınan verinin kontrol bilgisine ulaşmasını mümkün kılar.
- Mimari bilgisi saldırı hazırlığında önemlidir
  - İşlemci komut seti, endian düzeni, işletim sistemi ve stack frame biçimi exploit kodunun çalışabilirliğini belirler.

### Kısa Tekrar Notları

- Control hijacking: uygulamanın kontrol akışını ele geçirme.

- Buffer overflow en temel bellek taşması saldırısıdır.
- Stack yüksek adresten düşük adrese doğru büyür.
- Heap dinamik bellek tahsisi için kullanılır.
- Return address değiştirilebilirse program saldırgan koduna yönlendirilebilir.

## Detaylı Açıklamalar

- Kontrol akışını ele geçirme saldırılarında saldırgan, hedef programın beklediği girdiyi manipüle ederek programın başka bir kod parçasını çalıştırmasını sağlar. Bu kod parçası çoğu zaman shellcode veya benzeri küçük bir makine kodudur.
- Buffer overflow örneğinde yerel buffer için ayrılan alanın sınırı aşılr. Eğer sınır kontrolü yoksa fazla veri stack üzerindeki dönüş adresine kadar ilerleyebilir. Dönüş adresi saldırganın belirlediği adrese çevrildiğinde fonksiyon dönüşünde kontrol saldırgan koduna geçer.
- C/C++ ortamında pointer kullanımı, manuel bellek yönetimi, sınır kontrolünün programcıya bırakılması ve düşük seviyeli sistem kütüphanelerinin yaygınlığı bu saldırı sınıfını önemli hale getirir. Modern diller üst seviyede güvenli görünse bile çalışma zamanı ve sistem çağrılar bu düşük seviyeli bileşenlerle etkileşir.
- Bellek düzeninin bilinmesi saldırgan için değerlidir. Paylaşılan kütüphanelerin yüklenme bölgesi, executable başlangıcı, stack büyüme yönü ve endian düzeni exploit oluşturma sürecinde doğrudan kullanılır.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.